

Deepspace Defenders — Space Shooter Game

A Semester Project Report

Course: CSE142 Object Oriented Programming Techniques
Semester: Spring 2026
Instructor: Behraj Khan
Due Date: 10th May 2026
Author(s): Hamza Faisal — ERP: 32406
Hunain Adnan — ERP: 32397
Burhanuddin Murtaza Patanwala — ERP: 32400



Department of Computer Science
Institute of Business Administration, Karachi
May 10, 2026

Abstract

Deepspace Defenders is a fully playable 2D space shooter game developed in C++ using the SFML (Simple and Fast Multimedia Library) graphics framework. The player controls a spaceship and must survive waves of progressively harder enemies across three levels, each ending with a boss fight.

We built this game to apply the OOP concepts from our course in a real project. The architecture uses three inheritance hierarchies; a template class called `ObjectPool` to manage all live game objects without any dynamic allocation during gameplay; and encapsulation to protect classes' internal states. Score is added through an overloaded operator, and custom exception classes handle asset loading failures and invalid level transitions.

This results in a 60 fps arcade style space shooter game, which demonstrates the power of using OOP concepts effectively.

Keywords: Object-Oriented Programming, C++, SFML, Inheritance, Polymorphism, Encapsulation, Templates, Operator Overloading, Exception Handling, Recursion, Game Development.

Contents

Abstract	2
1 Introduction	5
1.1 Background	5
1.2 Problem Statement	5
1.3 Objectives	5
1.4 Scope and Limitations	5
2 OOP Design	6
2.1 Project File Structure	6
2.2 Class Hierarchy	6
2.3 Class and Struct Descriptions	7
2.3.1 Classes (with behaviour)	7
2.3.2 Data Structs (plain data containers)	9
2.4 OOP Concepts Summary	9
3 Implementation	11
3.1 Development Environment	11
3.2 Encapsulation — The Player Class	11
3.3 Inheritance — Three Class Hierarchies	11
3.3.1 Bullet Type Hierarchy	12
3.3.2 Enemy Type Hierarchy	13
3.3.3 Level Hierarchy	13
3.4 Polymorphism — Same Call, Different Behaviour	14
3.5 Templates — The ObjectPool	14
3.6 Operator Overloading	15
3.6.1 Player::operator+= (score addition)	15
3.6.2 GameState::operator== and operator!=	16
3.7 Exception Handling — Custom Exception Classes	16
3.8 Recursion — Layered Explosion System	17
3.9 Game Loop and State Machine	17
4 Testing	19
4.1 Test Cases	19
4.2 Memory Leak Testing	19
5 Expected Output — Screenshots	20
6 Results and Discussion	26
6.1 Results	26
6.2 Analysis	26
7 Challenges and Solutions	27
7.1 Pointer Leaks	27
7.2 Enemies All Spawning at the Same Position	27
7.3 Frame-rate-dependent Movement	27
8 Future Work	28
9 Conclusion	29

References	30
A Complete Source Code	31
A.1 GameData.h	31
A.2 Game.h	31
A.3 Level.h	33
A.4 Utils.h	33
A.5 BulletType.h	34
A.6 EnemyType.h	35
A.7 GameState.h	36
A.8 Player.h	36
A.9 main.cpp	37
A.10 Game_graphics.cpp (key excerpt)	38
A.11 Game.cpp (key excerpts)	38
B Build and Run Instructions	40
B.1 Prerequisites	40
B.2 Building with CMake	40
B.3 Running the Game	40
B.4 Controls	40

1 Introduction

1.1 Background

We chose to make a space shooter for our OOP project because everything in it — the player, enemies, bullets, levels, particles — can be implemented as a class. Each thing owns its own data and handles its own behaviour. The space shooter genre is simple enough to finish in time but complex enough to genuinely need every major OOP concept.

1.2 Problem Statement

We needed to build a game where:

- Multiple enemy types (small, medium, boss) share common logic but differ in stats and shooting patterns.
- Multiple levels each define their own wave structure but share common rendering and loading logic.
- Multiple bullet types (player, enemy, boss beam) are distinguishable without scattering `if/else` chains everywhere.
- Hundreds of live objects are managed without allocating or freeing memory every frame.
- The game had to flow through distinct screens (Home, Level Select, Playing, Paused, Game Over, Level Complete) in a predictable, bug-free way.

1.3 Objectives

1. Design three independent inheritance hierarchies (bullets, enemies, levels) that are easily extendable.
2. Implement a generic `ObjectPool` template that recycles objects in a fixed array, avoiding runtime memory allocation.
3. Encapsulate all player state inside the `Player` class so no other code can accidentally corrupt it.
4. Use virtual functions and abstract classes so that the main game loop does not need to know the concrete type of each object.
5. Demonstrate operator overloading, static class members, and custom exceptions as part of the design — not as add-ons.
6. Use a state stack to manage the different game states.

1.4 Scope and Limitations

In scope: Three levels, three enemy types, four bullet types, a particle explosion system, a scrolling star background, background music (OGG format via SFML Audio), a home screen, a level-select screen, pause/resume, game over, and level-complete overlays.

Out of scope: Saved high scores (no file I/O at runtime), multiplayer, sound effects, and power-ups. These are possible future extensions.

2 OOP Design

2.1 Project File Structure

The project has 12 source files. Each file only depends on files that appear above it in the list.

Table 1: Source files in dependency order

Layer	File	Purpose
1	<code>Utils.h</code>	Pure math helpers (vector length, normalise, lerp, clamp, random)
1	<code>BulletType.h</code>	Bullet category class + 4 concrete subclasses + <code>BulletTypeRegistry</code>
1	<code>EnemyType.h</code>	Enemy profile class + 3 concrete subclasses + <code>EnemyTypeRegistry</code>
1	<code>GameState.h</code>	Game-screen state value class (constants defined in <code>main.cpp</code>)
2	<code>GameData.h</code>	Data structs (Bullet, Enemy, Particle, Star, SpawnEvent, SpawnJob) + <code>ObjectPool</code> template
3	<code>Player.h</code>	Player class (fully inline, no <code>.cpp</code>)
4	<code>Level.h</code>	Abstract Level base class + <code>LevelOne</code> , <code>LevelTwo</code> , <code>LevelThree</code>
4	<code>Level.cpp</code>	Level method implementations and wave definitions
5	<code>Game.h</code>	Game class declaration
5	<code>Game.cpp</code>	Core logic: init, update, collision detection, enemy spawning
5	<code>Game_graphics.cpp</code>	All rendering methods (render, renderBullets, renderEnemies, HUD, overlays)
6	<code>main.cpp</code>	Defines all static member variables; instantiates registries; game entry point

2.2 Class Hierarchy

There are three inheritance hierarchies and one composition hub (`Game`). Figures 1 and 2 show the relationships.

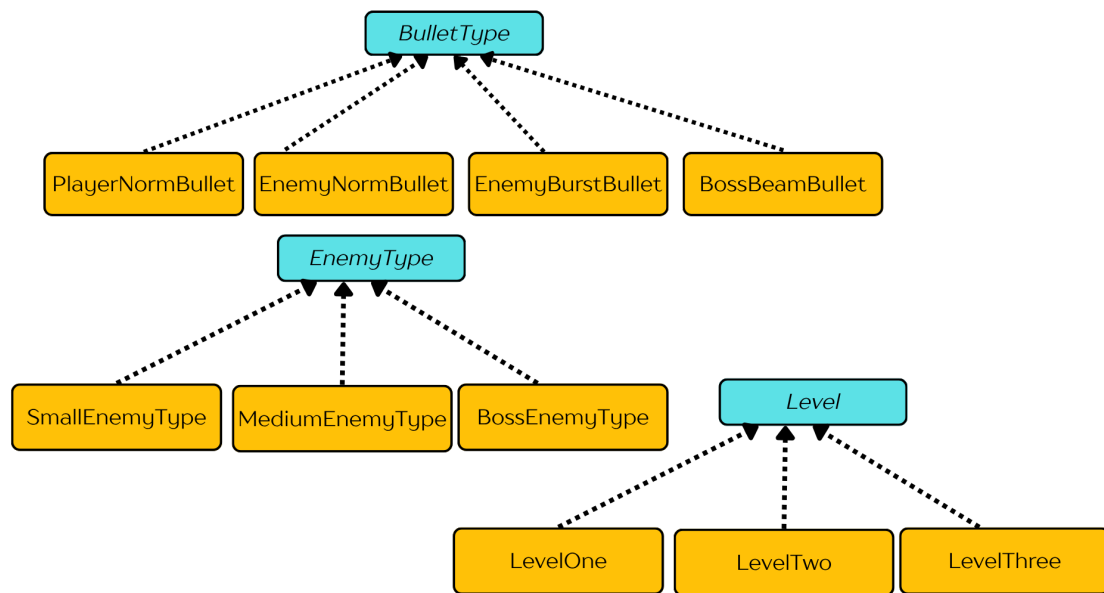


Figure 1: Three inheritance hierarchies (BulletType, EnemyType, Level). Arrows point from child to parent (“*inherits from*”). Shaded boxes are abstract bases; plain boxes are concrete subclasses.

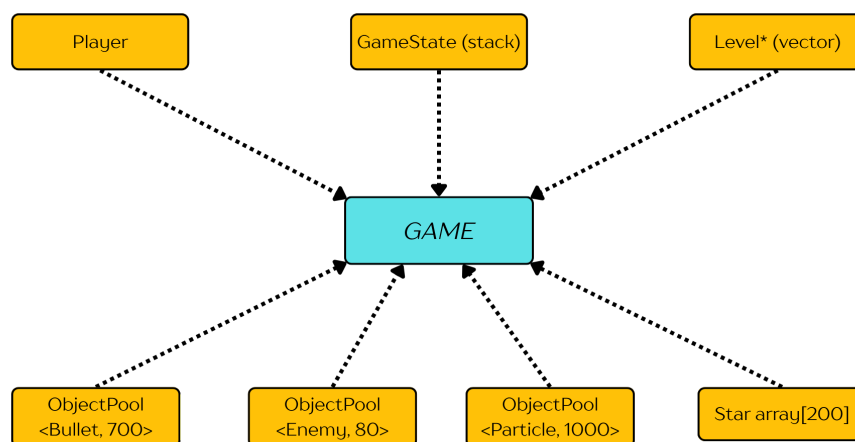


Figure 2: Composition: the **Game** class *owns* (has-a) all subsystems. Arrows labelled **has-a** mean “contains as a member variable”.

2.3 Class and Struct Descriptions

2.3.1 Classes (with behaviour)

Table 2: Classes and their responsibilities

Class	Responsibility	Key members
BulletType	Identifies a bullet's category and owner	id, playerBullet, getId(), isPlayerBullet(), static globals PlayerNorm, EnemyNorm, EnemyBurst, BossBeam
PlayerNormBullet	Player's shot	Inherits BulletType with id=0, playerBullet=true
EnemyNormBullet	Basic aimed enemy shot	id=3, playerBullet=false
EnemyBurstBullet	Spread-fire enemy shot	id=4, playerBullet=false
BossBeamBullet	Boss beam projectile	id=5, playerBullet=false
BulletTypeRegistry	Owns and initialises all four bullet-type pointers	Private members: one instance of each concrete subclass; constructor wires static pointers; destructor clears them to nullptr; copy disabled (= delete)
EnemyType	Stores the stat profile for one enemy category	maxHp, shootInterval, scoreValue, moveSpeed, radius, kind; static globals Small, Medium, Boss
SmallEnemyType	Fast, weak enemy	30 HP, interval 2.2s, radius 16px, worth 50 pts
MediumEnemyType	Tanky spread-shooter	80 HP, interval 1.8s, radius 22px, worth 120 pts
BossEnemyType	End-of-level boss	2500 HP, interval 0.8s, radius 60px, worth 2000 pts
EnemyTypeRegistry	Owns and initialises all three enemy-type pointers	Same pattern as BulletTypeRegistry; constructor assigns Small, Medium, Boss pointers; destructor sets them to nullptr; copy disabled
GameState	Tracks which screen is active	id; operator==, operator!=; static constants Home, LevelSelect, Playing, Paused, GameOver, LevelComplete
Player	Encapsulates all player state and behaviour	pos, hp, maxHp, speed, lives, score, timers; movement, clamping, damage, healing, score accumulation
Level	Abstract base for a playable level	name, desc, events (SpawnEvent list), planetTexture; pure virtual buildWaves(), getDuration(), getPlanetTexturePath()
LevelOne/Two/Three	Concrete level implementations	Override buildWaves() with a timed enemy schedule
ObjectPool<T,N>	Fixed-size object recycler	items[N]; alloc(), clearAll(), countActive()
Game	Central orchestrator	Owns all subsystems; implements init, run, update, collision, spawning; includes sf::Music for background audio; rendering split into Game_graphics.cpp

2.3.2 Data Structs (plain data containers)

Table 3: Data structs used as game objects

Struct	Represents	Key fields
Bullet	One active projectile	pos, vel, dmg, BulletType* type, on
Enemy	One active enemy	pos, vel, hp, maxHp, EnemyType* type, shootTimer, shootInterval, path, pathIndex, angle, pulse, scoreValue, moveSpeed, on
Particle	One explosion spark	pos, vel, colorStart, colorEnd, life, maxLife, size, on
SpawnEvent	A scheduled group spawn	time, etype, startPos, path, count, xSpacing, delay, fired
SpawnJob	One pending individual spawn	etype, startPos, delayRemaining, path, hp, maxHp, shootInterval, scoreValue, moveSpeed
Star	One background star	pos, speed, brightness, size

Why separate Struct from Class? We used structs for data-heavy objects like enemies to allow direct access to variables, which significantly improves performance and avoids the boilerplate of getters and setters.

2.4 OOP Concepts Summary

Table 4: OOP concepts used and where they appear

OOP Concept	Where and how it is used
Encapsulation	<code>Player</code> class: all fields are <code>private</code> ; external code uses getters and setters only
Inheritance	Three hierarchies: <code>BulletType</code> , <code>EnemyType</code> , <code>Level</code> (each with subclasses)
Polymorphism	<code>Level::buildWaves()</code> , <code>getDuration()</code> , and <code>getPlanetTexturePath()</code> are pure virtual; <code>Game</code> calls them without knowing the concrete type
Abstract Classes	<code>Level</code> cannot be instantiated directly because they have pure virtual methods
Templates	<code>ObjectPool<T,N></code> works for <code>Bullet</code> , <code>Enemy</code> , and <code>Particle</code> using the same code
Operator Overloading	<code>Player::operator+=(int)</code> adds score; <code>GameState::operator==</code> and <code>operator!=</code> compare states
Static Members	<code>BulletType::PlayerNorm</code> , <code>EnemyType::Small</code> , <code>GameState::Home</code> etc. — global pointers accessed through the class name
Exception Handling	<code>AssetLoadException</code> and <code>InvalidLevelException</code> (custom classes inheriting <code>std::runtime_error</code>)
Recursion	<code>spawnExplosion()</code> calls itself with <code>count/2</code> and <code>depth-1</code> ; base case is <code>depth <= 0</code>
Virtual Destructor	All parent classes use a virtual destructor to ensure correct cleanup of child classes, so derived objects are safely deleted via a base pointer

3 Implementation

3.1 Development Environment

- **Language:** C++17
- **Graphics & Audio library:** SFML 2.6 — graphics, window, and audio modules
- **Build system:** CMake 3.21+
- **IDE:** Visual Studio Code
- **Version control:** Git / GitHub
- **Platform:** Windows 11, MinGW-w64 (ucrt64)

3.2 Encapsulation — The Player Class

OOP Concept: Encapsulation

Encapsulation means keeping a class's data private and only letting outside code interact with it through a controlled public interface.

The `Player` class is the clearest example of encapsulation in this project. Every field — position, HP, lives, timers — is `private`. The rest of the code can only interact with the player through the public interface.

```
1 class Player {
2 private:
3     // position, hp, maxHp, speed, lives, score
4     // shootTimer, iframeTimer, shieldTimer, tilt, alive
5
6 public:
7     // Getters: getPos(), getHp(), getLives(), getScore(), isAlive(),
8     // ...
9     void takeDamage(float d);           // reduces hp
10    void heal(float h);                 // increases hp, capped at maxHp
11    void loseLife();                   // decrements lives
12    void clampToArena();               // keeps ship inside the play
13    // area
14    Player& operator+=(int bonus);     // adds score (operator overload)
15 };
```

Listing 1: Player class skeleton (Player.h)

Why this matters: If `hp` were public, any part of the code could accidentally write `player.hp = -999`. Making it `private` turns that into a compile error. Only `takeDamage()` and `heal()` can modify health, and both do it safely.

3.3 Inheritance — Three Class Hierarchies

OOP Concept: Inheritance

Inheritance lets a child class reuse the code of a parent class and then add or override specific behaviour. The parent defines the shared interface; each child fills in the parts that are different.

3.3.1 Bullet Type Hierarchy

We need to tell four kinds of bullets apart at runtime without scattering raw integer comparisons through the code. Each bullet type is its own class:

```

1  class BulletType {                      // abstract base -- defines the interface
2      int id; bool playerBullet;
3  public:
4      virtual ~BulletType() {}
5      int getId() const; bool isPlayerBullet() const;
6      static BulletType *PlayerNorm, *EnemyNorm, *EnemyBurst, *BossBeam
7      ;
8  };
9  // Concrete subclasses -- each hard-codes its own id
10 class PlayerNormBullet : public BulletType { /* id=0, player bullet
11     */ };
12 class EnemyNormBullet : public BulletType { /* id=3, enemy bullet
13     */ };
14 class EnemyBurstBullet : public BulletType { /* id=4, burst bullet
15     */ };
16 class BossBeamBullet : public BulletType { /* id=5, boss beam
17     */ };

```

Listing 2: BulletType hierarchy skeleton (BulletType.h)

The four concrete objects are owned by a BulletTypeRegistry class. The registries are local variables at the top of main(), created before the Game object, so the static pointers are valid for the entire program lifetime:

```

1  class BulletTypeRegistry {
2  private:
3      PlayerNormBullet playerNorm;        // owned by value -- no heap
4      EnemyNormBullet enemyNorm;          // allocation
5      EnemyBurstBullet enemyBurst;
6      BossBeamBullet bossBeam;
7  public:
8      BulletTypeRegistry() {
9          BulletType::PlayerNorm = &playerNorm;    // wire static
10         BulletType::EnemyNorm   = &enemyNorm;     // pointers
11         BulletType::EnemyBurst  = &enemyBurst;
12         BulletType::BossBeam    = &bossBeam;
13     }
14     ~BulletTypeRegistry() {
15         BulletType::PlayerNorm = nullptr;         // safe teardown at
16         BulletType::EnemyNorm  = nullptr;         // exit
17         BulletType::EnemyBurst = nullptr;
18         BulletType::BossBeam   = nullptr;
19     }
20     BulletTypeRegistry(const BulletTypeRegistry&) = delete;
21     BulletTypeRegistry& operator=(const BulletTypeRegistry&) = delete;
22 };

```

Listing 3: BulletTypeRegistry — RAII singleton manager (BulletType.h)

```

1  BulletType *BulletType::PlayerNorm = nullptr;
2  // ... (EnemyType and GameState statics defined here too)
3
4  int main() {
5      BulletTypeRegistry registryBullet; // wires BulletType statics
6      EnemyTypeRegistry  registry;      // wires EnemyType statics
7      try {
8          Game game; game.init(); game.run();
9      } catch (...) { ... }
10     // registries destroyed here -- destructors clear statics to
11         nullptr
12 }

```

Listing 4: Static definitions and registry activation in main.cpp

Why consolidate into main.cpp? C++ requires each static member variable to be defined in exactly one translation unit. Putting them all in main.cpp makes ownership clear and eliminates separate BulletType.cpp, EnemyType.cpp, and GameState.cpp files.

3.3.2 Enemy Type Hierarchy

Enemy types follow the same pattern but carry meaningful gameplay data:

```

1  class EnemyType { // abstract base -- stores gameplay stats
2      // maxHp, shootInterval, scoreValue, moveSpeed, radius, kind
3  public:
4      virtual ~EnemyType() {}
5      // Getters: getMaxHp(), getShootInterval(), getScoreValue(), ...
6      static EnemyType *Small, *Medium, *Boss;
7  };
8  // Stats baked in: (hp, shootInterval, score, speed, radius, kind)
9  class SmallEnemyType : public EnemyType { /* 30 HP, 2.2s, 50 pts */ };
10 class MediumEnemyType : public EnemyType { /* 80 HP, 1.8s, 120 pts */ };
11 class BossEnemyType : public EnemyType { /* 2500 HP, 0.8s, 2000 pts */ };

```

Listing 5: EnemyType hierarchy skeleton (EnemyType.h)

EnemyType.h also defines an EnemyTypeRegistry that follows the exact pattern as BulletTypeRegistry.

3.3.3 Level Hierarchy

```

1  class Level { // abstract base
2  protected:
3      // name, desc, events (wave schedule), planetTexture
4  public:
5      virtual ~Level() {}
6      // Pure virtual -- every subclass MUST implement:
7      virtual float getDuration() const = 0;
8      virtual std::string getPlanetTexturePath() const = 0;
9      virtual void buildWaves() = 0;
10     virtual void renderPlanet(sf::RenderWindow& w); // has default impl.
11 };
12 class LevelOne : public Level { /* 55s, planet1.png */ };

```

```

13 class LevelTwo    : public Level { /* 65s, planet2.png */ };
14 class LevelThree  : public Level { /* 80s, planet3.png, two bosses */
    };

```

Listing 6: Level abstract base class skeleton (Level.h)

3.4 Polymorphism — Same Call, Different Behaviour

OOP Concept: Polymorphism

If you have a base-class pointer and call a `virtual` function through it, C++ automatically calls the correct overridden version in the derived class — even though the pointer only knows about the base type. This is the use of polymorphism.

In `Game.cpp`, levels are stored as base-class pointers. The game calls virtual functions through those pointers without ever checking which concrete level is loaded:

```

1  std::vector<Level *> levels;    // stores LevelOne*, LevelTwo*, or
    LevelThree*
2
3  // During initialisation:
4  for (int i = 0; i < levels.size(); i++) {
5      levels[i]->buildWaves();    // calls the correct override
    automatically
6      levels[i]->loadPlanet();
7  }
8
9  // During gameplay rendering:
10 void Game::renderBackground() {
11     levels[currentLevel]->renderPlanet(window); // correct planet
    drawn automatically
12 }

```

Listing 7: Polymorphic level calls in Game.cpp

3.5 Templates — The ObjectPool

OOP Concept: Templates

A template is a class or function where the type is a parameter. You write the logic once, and the compiler generates a correctly-typed version for each type you use it with. This avoids writing the same container multiple times for different types.

The game can have up to 700 bullets, 80 enemies, and 1000 particles active at the same time. Calling `new` and `delete` every frame would be slow and could fragment memory. Instead we use a fixed-size *object pool*: a pre-allocated array where inactive slots are reused instead of freed.

```

1  template <class T, int N>
2  class ObjectPool
3  {
4  private:
5      std::array<T, N> items;    // fixed array -- allocated once, never
    reallocated
6
7  public:
8      ObjectPool() {

```

```

9         for (int i = 0; i < N; i++)
10             items[i].on = false;    // all slots start as "free"
11     }
12
13     int capacity() const { return N; }
14     T& at(int i)          { return items[i]; }
15
16     // Find the first free slot and return a pointer to it
17     T* alloc() {
18         for (int i = 0; i < N; i++) {
19             if (!items[i].on)
20                 return &items[i];
21         }
22         return nullptr;    // pool is full -- caller must handle this
23     }
24
25     void clearAll()          { for (int i = 0; i < N; i++) items[i].on =
26                             false; }
27     int countActive() const {
28         int c = 0;
29         for (int i = 0; i < N; i++) if (items[i].on) c++;
30         return c;
31     };

```

Listing 8: ObjectPool template class (GameData.h)

```

1 ObjectPool<Bullet,    MAX_BULLETS>    bullets;    // 700 slots
2 ObjectPool<Enemy,    MAX_ENEMIES>    enemies;    // 80 slots
3 ObjectPool<Particle, MAX_PARTICLES> particles;    // 1000 slots

```

Listing 9: Three pool instances in Game.h

The template is written once; the compiler generates three separate typed arrays from it. When a bullet is needed, `bullets.alloc()` finds a free slot and returns a pointer. When the bullet goes off-screen, `b.on = false` marks the slot as available again. No `new` or `delete` is called during gameplay.

3.6 Operator Overloading

OOP Concept: Operator Overloading

C++ lets you define what standard operators (+, =, ==, etc.) mean for your own classes. It makes code easier to read when used carefully.

3.6.1 Player::operator+= (score addition)

```

1 Player& operator+=(int scoreBonus) {
2     score += scoreBonus;
3     return *this;
4 }

```

Listing 10: Score addition via operator overload (Player.h)

Used in `Game.cpp` when an enemy is destroyed:

```

1 if (e.hp <= 0.f) {
2     e.on = false;
3     player += e.scoreValue;    // clean, readable syntax
4 }

```

Listing 11: Using the overloaded operator in Game.cpp

3.6.2 GameState::operator== and operator!=

```

1 bool operator==(const GameState& o) const { return id == o.id; }
2 bool operator!=(const GameState& o) const { return id != o.id; }

```

Listing 12: Comparison operators on GameState (GameState.h)

Used throughout Game.cpp for readable state checks:

```

1 GameState current = stateStack.top();
2 if (current == GameState::Playing) { update(dt); }

```

Listing 13: State comparison in Game.cpp

3.7 Exception Handling — Custom Exception Classes

OOP Concept: Exception Handling

Exceptions let you signal that something went wrong without scattering error checks through every function. When an error occurs you **throw** an exception and it propagates up the call stack until a **catch** block handles it.

We defined two custom exception classes inside Game.cpp:

```

1 // Thrown when a texture, font, or other asset cannot be loaded from
   // disk
2 class AssetLoadException : public std::runtime_error {
3 public:
4     AssetLoadException() : std::runtime_error("Failed to load asset")
5     {}
6 };
7 // Thrown when startLevel() is called with an invalid index
8 class InvalidLevelException : public std::runtime_error {
9 public:
10     InvalidLevelException() : std::runtime_error("Invalid level") {}
11 };

```

Listing 14: Custom exception classes (Game.cpp)

```

1 // In Game::init() -- missing assets are non-fatal; game warns and
   // continues
2 try {
3     loadTextureOrThrow(shipTexture, "assets/textures/spaceship.png");
4     // ... more assets ...
5 } catch (const AssetLoadException& ex) {
6     std::cerr << "Asset warning: " << ex.what() << "\n";
7 }
8
9 // In Game::startLevel() -- an invalid index is a programming bug

```



```

10 void Game::startLevel(int index) {
11     if (index < 0 || index >= (int)levels.size())
12         throw InvalidLevelException();
13 }
14
15 // In main.cpp -- the outermost safety net
16 int main() {
17     try { Game game; game.init(); game.run(); }
18     catch (const std::exception& ex) {
19         std::cerr << "Fatal error: " << ex.what() << std::endl;
20         return 1;
21     }
22 }

```

Listing 15: Throwing and catching exceptions (Game.cpp)

3.8 Recursion — Layered Explosion System

OOP Concept: Recursion

Recursion is when a function calls itself with a smaller version of the same problem until a base case is reached. Here, spawning an explosion that itself spawns smaller explosions is a natural fit.

When an enemy is destroyed, `Game::spawnExplosion()` spawns a burst of particles, then calls itself three times with `count/2` and `depth-1` to create sub-explosions at nearby random offsets:

```

1 void Game::spawnExplosion(sf::Vector2f pos, sf::Color cStart,
2                           sf::Color cEnd, int count, int depth) {
3     if (depth <= 0 || count <= 0) return; // base case
4
5     for (int i = 0; i < count; i++) {
6         Particle* p = particles.alloc();
7         if (!p) return;
8         // ... set pos, vel, color, life ...
9         p->on = true;
10    }
11    // Recursive step: three smaller explosions at random nearby
    offsets
12    for (int i = 0; i < 3; i++) {
13        sf::Vector2f offset(randFloat(-25.f, 25.f), randFloat(-25.f,
14        25.f));
15        spawnExplosion(pos + offset, cStart, cEnd, count / 2, depth -
16        1);
17    }
18 }

```

Listing 16: Recursive explosion spawner (Game.cpp)

3.9 Game Loop and State Machine

The `GameState` class works with a `std::stack` to implement a push/pop state machine. When the player pauses, the Paused state is pushed on top of Playing. When they resume, it is popped and Playing is active again. This means overlay states do not destroy the state underneath them.

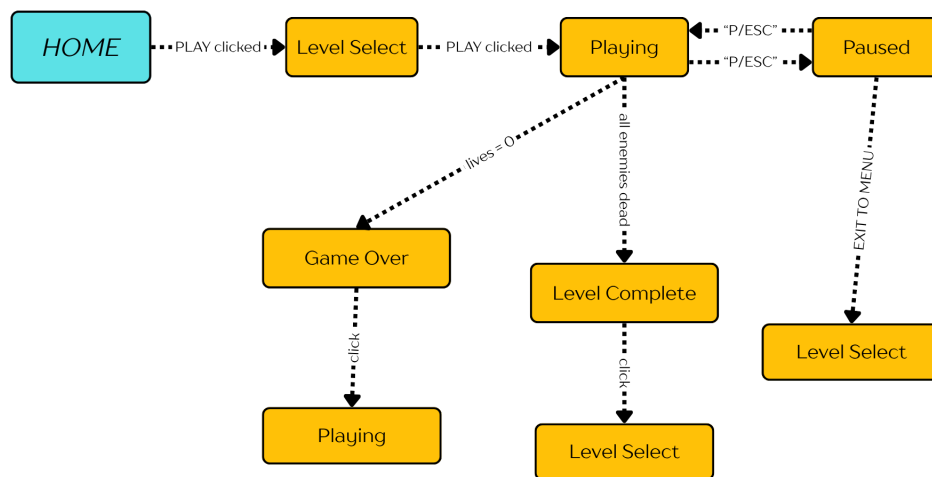


Figure 3: Game state transitions. Arrows show what action triggers each transition.

4 Testing

4.1 Test Cases

Table 5: Edge Case and Exception Test Cases

TC	Scenario	Expected Outcome	Pass?
TC-01	Hold Space to fire ~700 bullets	Pool fills up; <code>alloc()</code> returns <code>nullptr</code> ; no crash	✓
TC-02	Take damage immediately after a previous hit (iframes active)	Damage ignored for iframe duration; HP unchanged	✓
TC-03	Rename or delete a texture file; launch game	<code>AssetLoadException</code> caught; warning printed; game continues	✓
TC-04	Call <code>startLevel(-1)</code> programmatically	<code>InvalidLevelException</code> thrown and caught in <code>main()</code>	✓
TC-05	Kill all enemies including boss	<code>isLevelClear()</code> returns true; Level Complete overlay shown	✓

4.2 Memory Leak Testing

An earlier version used `new` to create the `BulletType` and `EnemyType` singletons. Since `delete` was never called on them, memory analysis tools reported leaks. We fixed this by introducing `BulletTypeRegistry` and `EnemyTypeRegistry`. Each registry stores its instances as plain member variables (no heap allocation), wires the static pointers in its constructor, and resets them to `nullptr` in its destructor. Both registries are local variables in `main()` and are cleaned up automatically when `main()` returns.

5 Expected Output — Screenshots

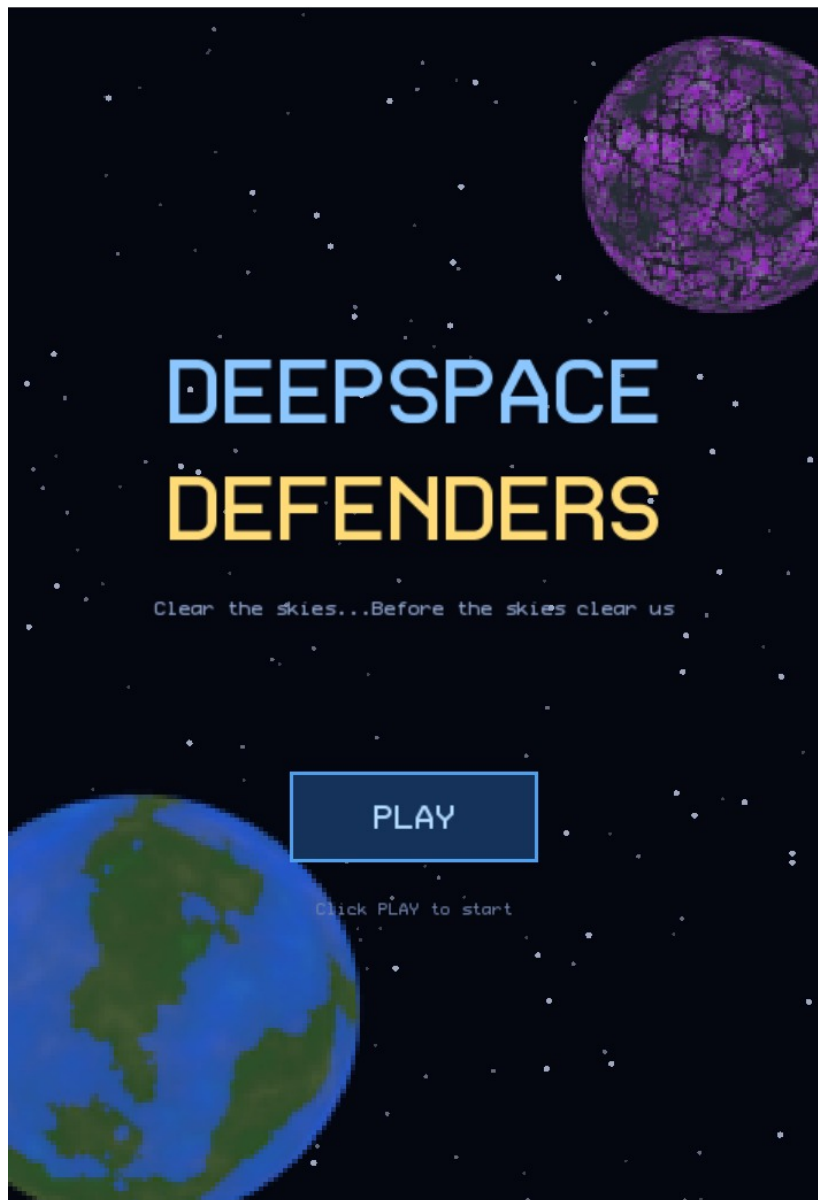


Figure 4: Home screen: title text, scrolling star background, two decorative planets, and the PLAY button.

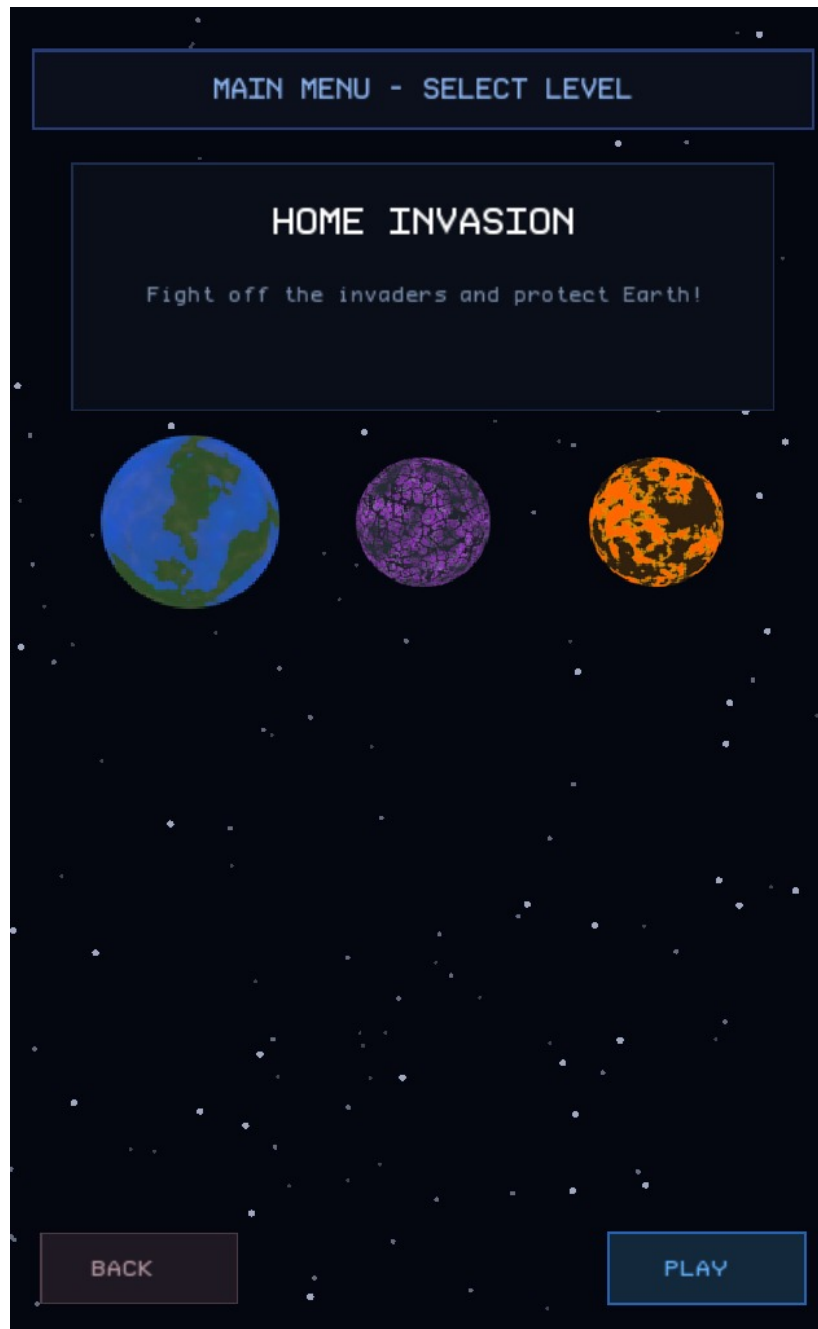


Figure 5: Level Select screen: three planet buttons (the selected one is larger), level name and description preview, BACK and PLAY buttons.

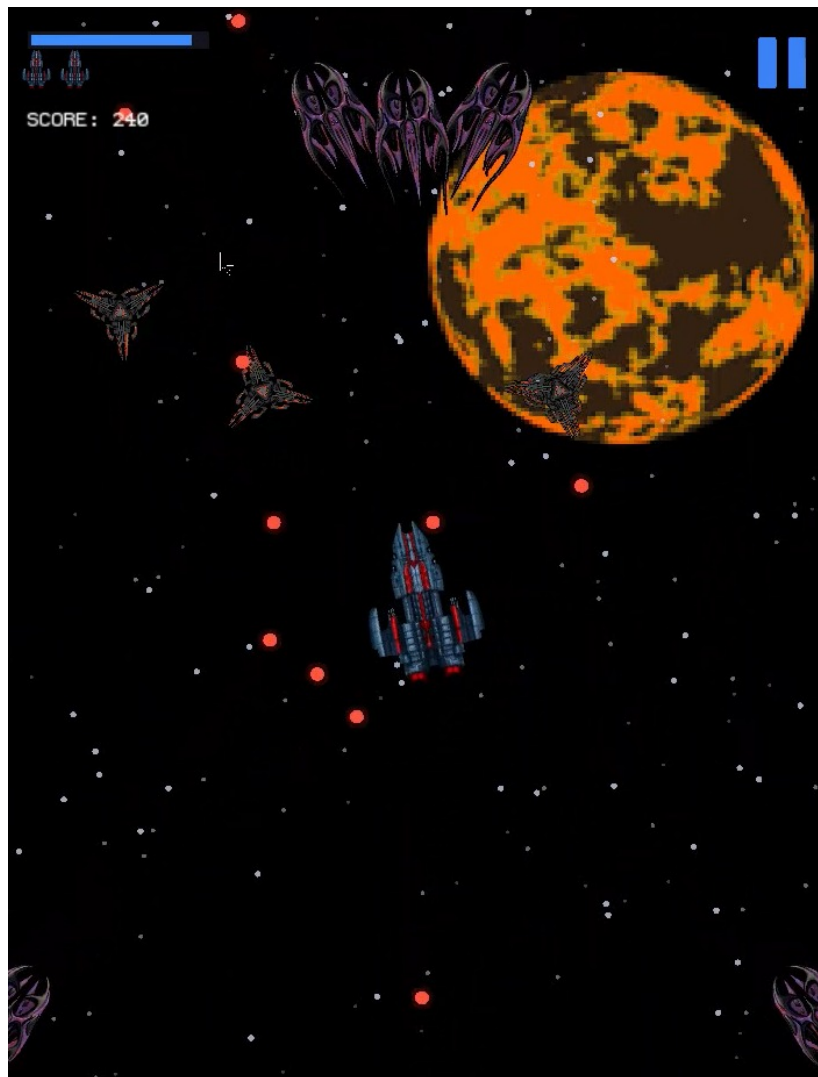


Figure 6: Mid-game: player ship, enemy formation following waypoints, player bullets, HUD showing HP bar, lives, and score.



Figure 7: Boss encounter: large boss sprite, red boss HP bar at the top, particle explosions visible.



Figure 8: Game Over overlay: dimmed background, “GAME OVER” heading, final score, and “Click to restart” prompt.

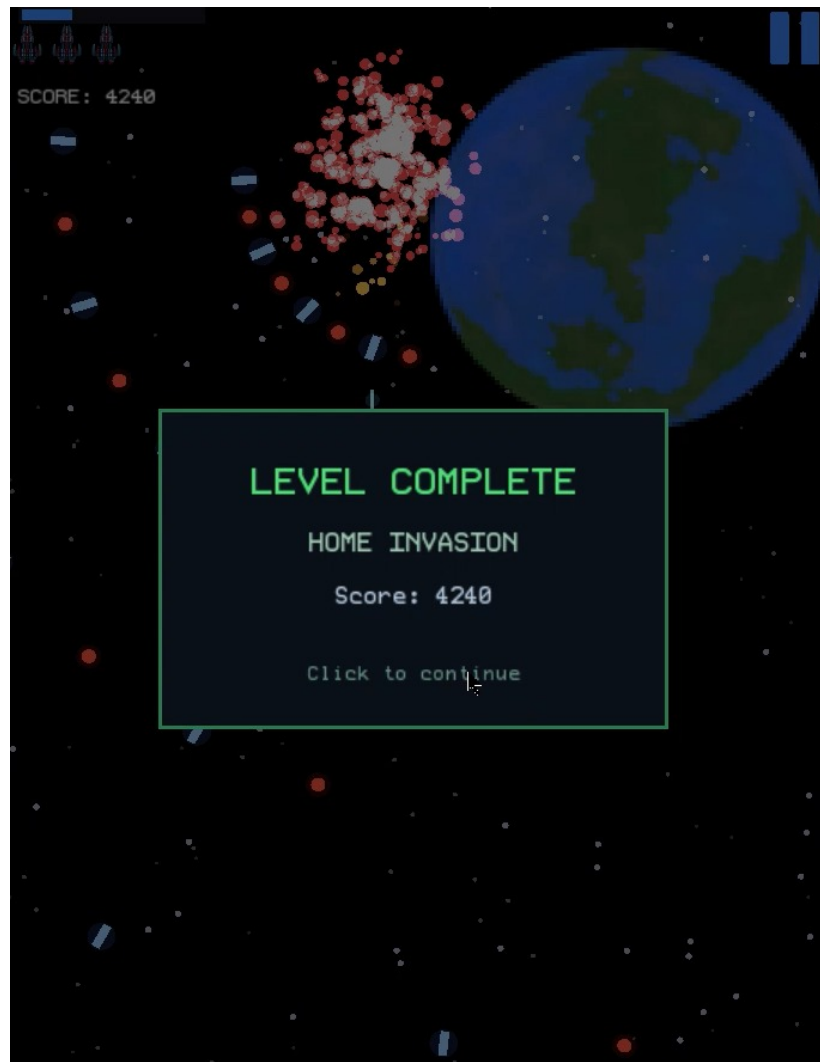


Figure 9: Level Complete overlay: green panel with level name, score, and “Click to continue” prompt.

6 Results and Discussion

6.1 Results

- The game runs at 60 fps on standard hardware.
- All three levels are completable.
- Background music plays via `sf::Music` using OGG audio streaming.
- The `ObjectPool` prevents runtime allocation: no `new` or `delete` occurs during gameplay.
- The state machine handles all screen transitions cleanly, including nested states (Paused inside Playing).
- Rendering logic is fully separated into `Game_graphics.cpp`, keeping `Game.cpp` focused on game logic only.

6.2 Analysis

- **ObjectPool vs dynamic allocation:** Searching a 700-slot array to find a free bullet takes minimal CPU time. The speed gained by skipping dynamic memory allocation makes this linear search highly efficient.
- **Polymorphism overhead:** Virtual functions add a tiny bit of processing time, but because we only use them for high-level tasks (like rendering the current level), the overhead is completely unnoticeable.
- **Template code size:** The compiler generates three separate versions of `ObjectPool`. For three small instantiations the binary size increase is negligible.

7 Challenges and Solutions

7.1 Pointer Leaks

Problem: The first version used `new` for global pointers; `delete` was never called, so memory tools reported leaks, even though the game was working.

Solution: Introduced registry classes that store instances by value and clear static pointers to `nullptr` in the destructor. Copy construction and assignment are = `deleted`.

Learning: managing dangling pointers manually and learning about the order of destructors called in main.

7.2 Enemies All Spawning at the Same Position

Problem: All enemies in a wave appeared at the same (x, y) .

Solution: We added spacing logic that shifts each enemy slightly to the left or right based on their index in the group.

Learning: Basic math offsets are essential when generating multiple game objects.

7.3 Frame-rate-dependent Movement

Problem: Objects moved a fixed number of pixels per frame.

Solution: All movement and timer updates are multiplied by `dt` (delta time in seconds). The product gives the same distance per second regardless of frame rate.

Learning: Always measure game speed in units per *second*, not per *frame*, to account for slower devices.

8 Future Work

- **Power-ups:** Add a `PowerUp` base class with subclasses
- **High score persistence:** Save the player's score to a `.txt` file on game over
- **Sound effects:** Use sounds for shoot, explosion, and level-complete sounds.
- **More levels:** Adding `LevelFour` only requires a new subclass of `Level` — no changes to `Game.cpp`.

9 Conclusion

Deepspace Defenders proves that OOP concepts aren't just academic theory; they are practical tools for solving real software problems. Encapsulation prevented bugs with player health. Inheritance and Polymorphism stopped us from writing massive if/else blocks for every enemy. Templates allowed us to write one memory manager and use it for bullets, enemies, and particles. By applying these concepts, we built a stable, fast, and organized C++ game.

References

- [1] B. Stroustrup, *The C++ Programming Language*, 4th ed. Addison-Wesley, 2013.
- [2] S. Meyers, *Effective Modern C++*. O'Reilly Media, 2014.
- [3] *SFML Documentation*. [Online]. Available: <https://www.sfml-dev.org/documentation/>. [Accessed: May 10, 2026].
- [4] *cppreference.com — C++ Reference*. [Online]. Available: <https://en.cppreference.com>. [Accessed: May 10, 2026].

A Complete Source Code

A.1 GameData.h

```

1  #pragma once
2  #include <SFML/Graphics.hpp>
3  #include <array>
4  #include <vector>
5  #include "BulletType.h"
6  #include "EnemyType.h"
7
8  struct Bullet { sf::Vector2f pos, vel; float dmg; BulletType* type;
9                 bool on; };
10 struct Enemy { sf::Vector2f pos, vel; float hp, maxHp; EnemyType*
11               type;
12               float shootTimer, shootInterval, angle, pulse,
13               moveSpeed;
14               int scoreValue, pathIndex; std::vector<sf::Vector2f>
15               > path; bool on; };
16 struct Particle { sf::Vector2f pos, vel; sf::Color colorStart,
17                  colorEnd;
18                  float life, maxLife, size; bool on; };
19 struct SpawnEvent { float time, xSpacing, delay; EnemyType* etype;
20                    sf::Vector2f startPos; std::vector<sf::Vector2f>
21                    path;
22                    int count; bool fired; };
23 struct SpawnJob { EnemyType* etype; sf::Vector2f startPos;
24                  float delayRemaining, hp, maxHp, shootInterval,
25                  moveSpeed;
26                  int scoreValue; std::vector<sf::Vector2f> path;
27                  };
28 struct Star { sf::Vector2f pos; float speed, brightness, size; };
29
30 const int MAX_BULLETS=700, MAX_ENEMIES=80, MAX_PARTICLES=1000,
31          MAX_STARS=200;
32
33 template <class T, int N>
34 class ObjectPool {
35     std::array<T, N> items;
36 public:
37     ObjectPool() { for (int i=0; i<N; i++) items[i].on = false; }
38     int capacity() const { return N; }
39     T& at(int i) { return items[i]; }
40     T* alloc() { for (int i=0; i<N; i++) if (!items[i].on) return &
41                 items[i]; return nullptr; }
42     void clearAll() { for (int i=0; i<N; i++) items[i].on = false; }
43     int countActive() const { int c=0; for (int i=0; i<N; i++) if (
44         items[i].on) c++; return c; }
45 };

```

Listing 17: GameData.h — all data structs and ObjectPool template

A.2 Game.h

```

1  #pragma once
2  #include <SFML/Graphics.hpp>
3  #include <SFML/Audio.hpp>
4  #include <stack>

```

```

5  #include <vector>
6  #include <queue>
7  #include "GameState.h"
8  #include "Player.h"
9  #include "GameData.h"
10 #include "Level.h"
11 #include "Utils.h"
12
13 class Game {
14 private:
15     sf::RenderWindow window;
16     sf::Font font;
17     sf::Music music;
18     sf::Texture shipTexture, smallEnemyTexture, mediumEnemyTexture,
19         bossEnemyTexture, homeplanet1, homeplanet2,
20         homeplanet3, pauseIcon;
21     sf::Sprite shipSprite, pauseSprite;
22     sf::View gameView;
23
24     std::stack<GameState> stateStack;
25     Player player;
26     ObjectPool<Bullet, MAX_BULLETS> bullets;
27     ObjectPool<Enemy, MAX_ENEMIES> enemies;
28     ObjectPool<Particle, MAX_PARTICLES> particles;
29     std::array<Star, MAX_STARS> stars;
30     std::vector<Level*> levels;
31     std::queue<SpawnJob> spawnQueue;
32     int currentLevel, selectedLevel;
33     float levelTimer, bgY;
34     bool bossActive;
35
36     void handleEvents(); void update(float dt); void render();
37     void startLevel(int index); void resetPlayer();
38     void checkCollisions(); void checkSpawns();
39     void spawnPlayerBullet(); void spawnEnemyBullet(Enemy& e);
40     void spawnEnemyFromJob(const SpawnJob& job);
41     void spawnExplosion(sf::Vector2f pos, sf::Color cs, sf::Color ce,
42         int count, int depth=1);
43     void loadFontOrThrow(const std::string& path);
44     void loadTextureOrThrow(sf::Texture& tex, const std::string& path
45         );
46
47     void renderBackground(); void renderStars(); void renderPlayer();
48     void renderEnemies(); void renderBullets(); void renderParticles
49         ();
50     void renderHUD(); void renderBossHP();
51     void renderHomeScreen(); void renderLevelSelect();
52     void renderPauseOverlay(); void renderGameOverOverlay();
53     void renderLevelCompleteOverlay();
54     void drawPlayerShip(sf::Vector2f pos, float tilt, float scale);
55     void drawSmallEnemy(sf::Vector2f pos, float angle);
56     void drawMediumEnemy(sf::Vector2f pos, float angle);
57     void drawBossEnemy(sf::Vector2f pos, float angle);
58     void initStars(); void buildLevels();
59     void drawTextCentered(const std::string& str, float y, int size,
60         sf::Color col);
61     bool isLevelClear(); int countActiveEnemies();

```



```

59 public:
60     Game(); ~Game();
61     void init(); void run();
62 };

```

Listing 18: Game.h — Game class declaration

A.3 Level.h

```

1  #pragma once
2  #include <SFML/Graphics.hpp>
3  #include <string>
4  #include <vector>
5  #include "GameData.h"
6
7  class Level {
8  protected:
9      std::string name, desc;
10     std::vector<SpawnEvent> events;
11     sf::Texture planetTexture;
12     bool planetLoaded;
13 public:
14     Level() : planetLoaded(false) {}
15     virtual ~Level() {}
16     std::string getName() const { return name; }
17     std::string getDesc() const { return desc; }
18     std::vector<SpawnEvent>& getEvents() { return events; }
19     virtual float getDuration() const = 0;
20     virtual std::string getPlanetTexturePath() const = 0;
21     virtual void buildWaves() = 0;
22     virtual void renderPlanet(sf::RenderWindow& window);
23     void loadPlanet();
24 };
25
26 class LevelOne : public Level { public: LevelOne();
27     float getDuration() const override { return 55.f; }
28     std::string getPlanetTexturePath() const override;
29     void buildWaves() override; };
30
31 class LevelTwo : public Level { public: LevelTwo();
32     float getDuration() const override { return 65.f; }
33     std::string getPlanetTexturePath() const override;
34     void buildWaves() override; };
35
36 class LevelThree : public Level { public: LevelThree();
37     float getDuration() const override { return 80.f; }
38     std::string getPlanetTexturePath() const override;
39     void buildWaves() override; };

```

Listing 19: Level.h — abstract Level base and concrete subclasses

A.4 Utils.h

```

1  #pragma once
2  #include <SFML/Graphics.hpp>
3  #include <cmath>
4  #include <cstdlib>

```

```

5
6 inline float vlen(sf::Vector2f v) { return std::sqrt(v.x*v.x + v.y*v.y); }
7 inline sf::Vector2f vnorm(sf::Vector2f v) {
8     float l = vlen(v);
9     return (l < 0.0001f) ? sf::Vector2f(0.f,0.f) : sf::Vector2f(v.x/l
10         , v.y/l);
11 }
12 inline float lerp(float a, float b, float t) { return a + (b-a)*t; }
13 inline sf::Color colorLerp(sf::Color a, sf::Color b, float t) {
14     t = (t<0.f)?0.f:(t>1.f?1.f:t);
15     return sf::Color((sf::Uint8)(a.r+(b.r-a.r)*t), (sf::Uint8)(a.g+(b
16         .g-a.g)*t),
17         (sf::Uint8)(a.b+(b.b-a.b)*t), (sf::Uint8)(a.a+(b
18         .a-a.a)*t));
19 }
20 inline float clampf(float v, float lo, float hi) {
21     return (v<lo)?lo:(v>hi?hi:v); }
22 inline float randFloat(float lo, float hi) {
23     return lo + (float)rand()/(float)RAND_MAX * (hi-lo); }
24 inline int randInt(int lo, int hi) {
25     return (hi<=lo)?lo : lo + rand()%(hi-lo+1); }

```

Listing 20: Utils.h — math utility functions

A.5 BulletType.h

```

1 #pragma once
2
3 class BulletType {
4     int id;
5     bool playerBullet;
6 public:
7     BulletType(int id, bool playerBullet) : id(id), playerBullet(
8         playerBullet) {}
9     virtual ~BulletType() {}
10    int getId() const { return id; }
11    bool isPlayerBullet() const { return playerBullet; }
12    static BulletType *PlayerNorm, *EnemyNorm, *EnemyBurst, *BossBeam
13    ;
14 };
15 class PlayerNormBullet : public BulletType {
16 public: PlayerNormBullet() : BulletType(0, true) {} };
17 class EnemyNormBullet : public BulletType {
18 public: EnemyNormBullet() : BulletType(3, false) {} };
19 class EnemyBurstBullet : public BulletType {
20 public: EnemyBurstBullet() : BulletType(4, false) {} };
21 class BossBeamBullet : public BulletType {
22 public: BossBeamBullet() : BulletType(5, false) {} };
23
24 class BulletTypeRegistry {
25     PlayerNormBullet playerNorm;
26     EnemyNormBullet enemyNorm;
27     EnemyBurstBullet enemyBurst;
28     BossBeamBullet bossBeam;
29 public:
30     BulletTypeRegistry() {

```

```

29     BulletType::PlayerNorm = &playerNorm; BulletType::EnemyNorm
        = &enemyNorm;
30     BulletType::EnemyBurst = &enemyBurst; BulletType::BossBeam
        = &bossBeam;
31 }
32 ~BulletTypeRegistry() {
33     BulletType::PlayerNorm = nullptr; BulletType::EnemyNorm =
        nullptr;
34     BulletType::EnemyBurst = nullptr; BulletType::BossBeam =
        nullptr;
35 }
36 BulletTypeRegistry(const BulletTypeRegistry&) = delete
    ;
37 BulletTypeRegistry& operator=(const BulletTypeRegistry&) = delete
    ;
38 };

```

Listing 21: BulletType.h — bullet type hierarchy and registry

A.6 EnemyType.h

```

1  #pragma once
2
3  class EnemyType {
4      float maxHp, shootInterval, moveSpeed, radius;
5      int    scoreValue, kind;
6  public:
7      EnemyType(float maxHp, float shootInterval, int scoreValue,
8                float moveSpeed, float radius, int kind)
9          : maxHp(maxHp), shootInterval(shootInterval), scoreValue(
            scoreValue),
10            moveSpeed(moveSpeed), radius(radius), kind(kind) {}
11     virtual ~EnemyType() {}
12     float getMaxHp() const { return maxHp; }
13     float getShootInterval() const { return shootInterval; }
14     int  getScoreValue() const { return scoreValue; }
15     float getMoveSpeed() const { return moveSpeed; }
16     float getRadius() const { return radius; }
17     int  getKind() const { return kind; }
18     static EnemyType *Small, *Medium, *Boss;
19 };
20 class SmallEnemyType : public EnemyType {
21 public: SmallEnemyType() : EnemyType( 30.f, 2.2f, 50, 200.f, 16.f
    , 0) {} };
22 class MediumEnemyType : public EnemyType {
23 public: MediumEnemyType() : EnemyType( 80.f, 1.8f, 120, 160.f, 22.f
    , 1) {} };
24 class BossEnemyType : public EnemyType {
25 public: BossEnemyType() : EnemyType(2500.f, 0.8f, 2000, 80.f, 60.f
    , 2) {} };
26
27 class EnemyTypeRegistry {
28     SmallEnemyType small; MediumEnemyType medium; BossEnemyType boss;
29 public:
30     EnemyTypeRegistry() {
31         EnemyType::Small = &small; EnemyType::Medium = &medium;
            EnemyType::Boss = &boss;
32     }

```

```

33 ~EnemyTypeRegistry() {
34     EnemyType::Small = nullptr; EnemyType::Medium = nullptr;
35     EnemyType::Boss = nullptr;
36 }
36 EnemyTypeRegistry(const EnemyTypeRegistry&) = delete;
37 EnemyTypeRegistry& operator=(const EnemyTypeRegistry&) = delete;
38 };

```

Listing 22: EnemyType.h — enemy type hierarchy and registry

A.7 GameState.h

```

1  #pragma once
2  class GameState {
3      int id;
4  public:
5      GameState(int i = 0) : id(i) {}
6      int getId() const { return id; }
7      bool operator==(const GameState& o) const { return id == o.id; }
8      bool operator!=(const GameState& o) const { return id != o.id; }
9      static const GameState Home, LevelSelect, Playing, Paused,
10         GameOver, LevelComplete;
11 };

```

Listing 23: GameState.h — game screen state (constants defined in main.cpp)

A.8 Player.h

```

1  #pragma once
2  #include <SFML/Graphics.hpp>
3  #include "Utils.h"
4
5  class Player {
6  private:
7      sf::Vector2f pos;
8      float hp, maxHp, speed;
9      int lives, score;
10     float shootTimer, shootInterval, iframeTimer, shieldTimer, tilt;
11     bool alive;
12 public:
13     Player() : pos(240.f, 600.f), hp(100.f), maxHp(100.f), speed(310.f),
14         lives(3), score(0), shootTimer(0.f), shootInterval(0.10f),
15         iframeTimer(0.f), shieldTimer(0.f), tilt(0.f), alive(true) {}
16
17     sf::Vector2f getPos() const { return pos; }
18     float getHp() const { return hp; }
19     float getMaxHp() const { return maxHp; }
20     float getSpeed() const { return speed; }
21     int getLives() const { return lives; }
22     int getScore() const { return score; }
23     float getShootTimer() const { return shootTimer; }
24     float getShootInterval() const { return shootInterval; }
25     float getIframeTimer() const { return iframeTimer; }
26     float getShieldTimer() const { return shieldTimer; }

```

```

27     float      getTilt()          const { return tilt; }
28     bool       isActive()        const { return alive; }
29
30     void setPos(sf::Vector2f p)   { pos = p; }
31     void setTilt(float t)         { tilt = t; }
32     void setShootTimer(float t)   { shootTimer = t; }
33     void setIframeTimer(float t)  { iframeTimer = t; }
34     void setShieldTimer(float t)  { shieldTimer = t; }
35     void setAlive(bool a)         { alive = a; }
36
37     void move(sf::Vector2f d)     { pos += d; }
38     void clampToArena() {
39         pos.x = clampf(pos.x, 24.f, 456.f);
40         pos.y = clampf(pos.y, 24.f, 696.f);
41     }
42     void takeDamage(float d)      { hp -= d; }
43     void heal(float h)            { hp = (hp+h > maxHp) ? maxHp : hp+h; }
44     void loseLife()              { lives--; }
45     void tickShootTimer(float dt) { shootTimer -= dt; }
46     void tickIframe(float dt)    { iframeTimer -= dt; }
47     void tickShield(float dt)    { shieldTimer -= dt; }
48
49     void resetForLevel() {
50         pos = sf::Vector2f(240.f, 600.f);
51         hp = maxHp; speed = 310.f;
52         shootTimer = iframeTimer = shieldTimer = tilt = 0.f;
53         alive = true; score = 0; lives = 3;
54     }
55
56     Player& operator+=(int s) { score += s; return *this; }
57 };

```

Listing 24: Player.h — fully encapsulated player class

A.9 main.cpp

```

1  #include "Game.h"
2  #include <iostream>
3  #include "BulletType.h"
4  #include "EnemyType.h"
5  #include "GameState.h"
6
7  const GameState GameState::Home(0), GameState::LevelSelect(1),
8          GameState::Playing(2), GameState::Paused(3),
9          GameState::GameOver(4), GameState::LevelComplete(5);
10
11  BulletType *BulletType::PlayerNorm = nullptr, *BulletType::EnemyNorm
12      = nullptr,
13      *BulletType::EnemyBurst = nullptr, *BulletType::BossBeam
14      = nullptr;
15
16  EnemyType *EnemyType::Small = nullptr, *EnemyType::Medium = nullptr,
17      *EnemyType::Boss = nullptr;
18
19  int main() {
20      BulletTypeRegistry registryBullet;
21      EnemyTypeRegistry registry;
22      try {

```

```

21     Game game; game.init(); game.run();
22 } catch (const std::exception& ex) {
23     std::cerr << "Fatal error: " << ex.what() << std::endl;
24     return 1;
25 }
26 return 0;
27 }

```

Listing 25: main.cpp — static definitions, registry activation, and entry point

A.10 Game_graphics.cpp (key excerpt)

```

1 void Game::render() {
2     window.setView(window.getDefaultView());
3     window.clear(sf::Color::Black);
4     window.setView(gameView);
5     GameState current = stateStack.top();
6     if (current == GameState::Home) { renderHomeScreen();
7         return; }
8     if (current == GameState::LevelSelect){ renderLevelSelect();
9         return; }
10    renderBackground(); renderStars(); renderEnemies();
11    renderBullets(); renderParticles(); renderPlayer(); renderHUD();
12    if (bossActive) renderBossHP();
13    if (current == GameState::Paused) renderPauseOverlay();
14    if (current == GameState::GameOver) renderGameOverOverlay();
15    if (current == GameState::LevelComplete)
16        renderLevelCompleteOverlay();
17 }
18
19 void Game::renderBullets() {
20     for (int i = 0; i < bullets.capacity(); i++) {
21         Bullet& b = bullets.at(i);
22         if (!b.on || !b.type) continue;
23         if (b.type->isPlayerBullet()) {
24             sf::RectangleShape streak(sf::Vector2f(2.f, 12.f));
25             streak.setPosition(b.pos);
26             streak.setFillColor(sf::Color(150, 230, 255));
27             window.draw(streak);
28         } else if (b.type == BulletType::BossBeam) {
29             sf::RectangleShape beam(sf::Vector2f(5.f, 14.f));
30             float angle = std::atan2(b.vel.y, b.vel.x) * 180.f /
31                 3.14159f - 90.f;
32             beam.setRotation(angle);
33             beam.setFillColor(sf::Color(140, 180, 255));
34             window.draw(beam);
35         }
36         // else: enemy normal/burst bullets drawn as red orbs
37     }
38 }

```

Listing 26: Game_graphics.cpp — render() dispatcher and renderBullets() (excerpt)

A.11 Game.cpp (key excerpts)

```

1 class AssetLoadException : public std::runtime_error {
2 public: AssetLoadException() : std::runtime_error("Failed to load
3     asset") {} };

```

```

3 class InvalidLevelException : public std::runtime_error {
4 public: InvalidLevelException() : std::runtime_error("Invalid level")
    {} };
5
6 void Game::init() {
7     // ... window and viewport setup ...
8     try {
9         loadTextureOrThrow(shipTexture, "assets/textures/spaceship.
            png");
10        // ... more assets ...
11    } catch (const AssetLoadException& ex) {
12        std::cerr << "Asset warning: " << ex.what() << "\n";
13    }
14    if (!music.openFromFile("assets/audio/music1.ogg"))
15        std::cerr << "Music load failed\n";
16    else { music.setLoop(true); music.setVolume(50.f); music.play();
    }
17    buildLevels(); initStars();
18    stateStack.push(GameState::Home);
19 }
20
21 void Game::startLevel(int index) {
22     if (index < 0 || index >= (int)levels.size())
23         throw InvalidLevelException();
24     currentLevel = index;
25     // ... clear pools, reset timer, reset player ...
26 }
27
28 void Game::checkCollisions() {
29     for (int i = 0; i < bullets.capacity(); i++) {
30         Bullet& b = bullets.at(i);
31         if (!b.on || !b.type || !b.type->isPlayerBullet()) continue;
32         for (int j = 0; j < enemies.capacity(); j++) {
33             Enemy& e = enemies.at(j);
34             if (!e.on) continue;
35             if (vlen(b.pos - e.pos) < 4.f + e.type->getRadius()) {
36                 e.hp -= b.dmg; b.on = false;
37                 if (e.hp <= 0.f) {
38                     e.on = false;
39                     player += e.scoreValue; // operator+= overload
40                     spawnExplosion(e.pos, ...);
41                 }
42                 break;
43             }
44         }
45     }
46     // ... enemy bullet vs player collision ...
47 }

```

Listing 27: Game.cpp — custom exceptions, init, and collision sketch

B Build and Run Instructions

B.1 Prerequisites

- MinGW-w64 (ucrt64) on Windows — install SFML via:
pacman -S mingw-w64-ucrt-x86_64-sfml
- CMake 3.21 or later
- Visual Studio Code (or any editor)

B.2 Building with CMake

```

1 Using VS Code and CMake: press Ctrl+Shift+P and select "CMake:
  Configure",
2 then press Ctrl+Shift+P again and select "CMake: Build".
3 This produces the executable at build/SpaceShooter.exe

```

B.3 Running the Game

```

1 Navigate to the build/ folder and double-click SpaceShooter.exe to
  run the game.

```

B.4 Controls

Key / Input	Action
W / A / S / D or Arrow keys	Move ship
Space or Left mouse click	Shoot / navigate menus
P or Escape	Pause / Resume
Left mouse click	Navigate menus

Table 6: Player controls